

OBJECT ORIENTED PROGRAMMING USING C++

Course code: CAP455

Student name: SHAZIN ADINAN K P

REGISTRATION NO: 12625822

Roll No: 55

Faculty: MR. ADARSH KUMAR SINGH

CA-1

GitHub Repository: https://github.com/shazinadinan/Community-Water-Quality-Test-Manager.git

CA-1	**Project: Community Water Quality Test Manager** Increment 1: Registering water sources and samples, validated readings (manual or rand() simulated) stored in arrays of objects, with findings and summary report

APPENDIX / TABLE OF CONTENTS

SL.No.	Content	Page No.
1	Introduction	3
2	Project Objectives	3
3	Software and Concepts Used	3
4	System Design and Data Structure	5
5	Program Working	7
6	Explanation of Important Code	10
7	Testing and Expected Results	13
8	Current Scope and Limitations	14
9	Future Enhancements	15
10	Appendix A: Source Code	15
11	Conclusion	23

1. Introduction

2. Project Objectives

1. Register water sources (well, borewell, lake, river, tank) with a unique source ID, a name and a location chosen from a location tree.
2. Register water samples with a unique sample ID, a valid collection date and the readings pH, turbidity and hardness.
3. Allow readings to be typed manually or simulated with rand() in four scenarios; simulated data is always labelled.
4. Compare every reading with configurable example limits, set the sample status and show parameter-wise findings plus a combined example score (0 to 100).
5. Show all samples, one sample in detail, and a summary report with counts, flagged readings and averages by source type.
1. Validate every input (menu, numbers, dates, IDs) so that wrong or non-numeric entries never crash the program or create duplicate records.
2. Store the data in arrays of objects and organise the code into classes and functions so that it can be extended with constructors, files, inheritance and the STL in CA-2 to CA-4.

3. Software and Concepts Used

3.1 Software

Item	Details
Language	C++ (standard C++11 or later; written and tested as C++17)
Compiler	GNU g++ 13.3 on Ubuntu. Compiled with no warnings using -Wall -Wextra -pedantic in C++11, C++14, C++17 and C++20 modes.
Checking tools	Run with AddressSanitizer and UndefinedBehaviorSanitizer: no memory errors in the test runs.
IDE / Editor	Any C++ IDE such as Dev-C++, Code::Blocks or VS Code
Header files	<iostream> (cin, cout), <iomanip> (setw, fixed, setprecision), <string>, <cstdlib> (rand, srand, exit, atoi), <cctype> (isdigit), <ctime> (time)
Platform	Console application; runs on Windows, Linux and macOS

3.2 C++ Concepts Used (CA-1: Units I and II)

Concept	Where it is used in the program
Input/output streams	cin, cout and getline in readInt(), readDouble(), readLine(); setw, fixed and setprecision for formatted output
Classes and objects	class WaterSource, class WaterSample, class SummaryTable, class SampleAuditor
Enumerations	enum SourceType (WELL..TANK) and enum SampleStatus (PENDING, TESTED, ATTENTION)
Structures	struct Limits holds the example limits; struct Location holds a location name and its parent index

Concept	Where it is used in the program
Static members	static int totalSamples and static getTotalSamples() in WaterSample
Inline / non-inline	Getters such as getId() are defined inside the class (inline); display(), evaluate() are defined outside (non-inline)
Friend function / class	friend double combinedScore(const WaterSample&); friend class SampleAuditor
Function overloading	banner(title) and banner(title, width)
Reference variables	simulateReadings(int, double&, double&, double&); const string& parameters
Call by value / address / reference	Value: isLeap(int y). Address: clampToRange(double*). Reference: simulateReadings(int, double&, double&, double&)
Recursion	printLocationTree() and locationPath() walk the location hierarchy
Pointers	findSample() and findSource() return a pointer or nullptr; viewOne() uses p->displayFindings(); &samples[i] is the address of an object
this pointer	setBasic() and setReadings() return *this so calls can be chained
Arrays	WaterSource sources[10], WaterSample samples[50], Location locations[]
Multidimensional arrays	double sum[6][3] inside class SummaryTable (in a class) and int stat[3][6] in summaryReport() (in a function)
Array of objects	samples[MAX_SAMPLES] and sources[MAX_SOURCES]
Strings	std::string members and parameters; substr(), size() and + (concatenation) in validDate() and locationPath()
Loops	do-while for the menu; while(true) for input; for loops for lists and totals
switch-case	Main menu (1 to 10) and the four simulation scenarios
rand() / srand()	simulateReadings() makes random values; --seed makes a run repeatable

4. System Design and Data Structure

4.1 Data dictionary: classes and structures

Class WaterSample (main entity, unique key = sampleId)

Member	Type	Rule / purpose
sampleId	int	Unique key, 1 to 99999; duplicates are rejected
sourceId	int	Must match an existing WaterSource id
collectionDate	string	YYYY-MM-DD, a real calendar date (leap years checked), year 2000 to 2100
ph	double	0 to 14
turbidity	double	0 to 1000 NTU
hardness	double	0 to 2000 mg/L
status	SampleStatus	PENDING (default), TESTED or ATTENTION; recalculated by evaluate()
simulated	bool	true when the values came from rand()
totalSamples	static int	Number of samples created so far, shared by all objects

Class WaterSource

Member	Type	Rule / purpose
id	int	Unique key, 1 to 999
name	string	Not empty
type	SourceType	WELL, BOREWELL, LAKE, RIVER or TANK
locationIndex	int	Index 0 to 5 into the locations[] tree

Other structures and classes

Name	Kind	Purpose
Limits (gLimits)	struct	Example limits: phMin 6.5, phMax 8.5, turbidityMax 5.0 NTU, hardnessMax 300 mg/L; editable from the menu
Location	struct	name and parent index; 6 entries form a tree (Community > Ward > Area)
SummaryTable	class	double sum[6][3] and int count[6]: totals per source type for the averages
SampleAuditor	friend class	Reads private members of WaterSample to print an audit line

4.2 Main Data

- WaterSource sources[MAX_SOURCES] with sourceCount: array of up to 10 sources.
- WaterSample samples[MAX_SAMPLES] with sampleCount: array of up to 50 samples; the next sample is stored in samples[sampleCount].
- Limits gLimits: the one shared set of example limits.
- Location locations[]: the location tree used by recursion.
- WaterSample::totalSamples: static counter of samples created.

4.3 Functions

Function	Parameters / Return	Purpose
readInt / readDouble / readLine	prompt (and range) / value	Read input again and again until it is valid; stop at end of input
validDate / readDate	string / bool, string	Check YYYY-MM-DD and leap years
registerSource	none / void	Add a new source (limit and duplicate checks)
addSample	bool simulate / void	Ask ID, source, date, readings (manual or rand()) and store the sample
simulateReadings	int, double&, double&, double& / void	Generate readings for a scenario
findSample / findSource	int id / pointer	Return pointer to the record or nullptr
WaterSample::evaluate	none / void	Compare readings with gLimits and set status
WaterSample::displayFindings	none / void	Print parameter-wise findings

Function	Parameters / Return	Purpose
combinedScore (friend)	const WaterSample& / double	0 to 100 example score
summaryReport	none / void	Counts, flagged readings, averages by type
editLimits	none / void	Change limits and re-assess all samples
printLocationTree / locationPath	node, depth / void; node / string	Recursive location list and path
main	argc, argv / int	Seed rand(), run the menu with do-while and switch

4.4 UML Class Diagram

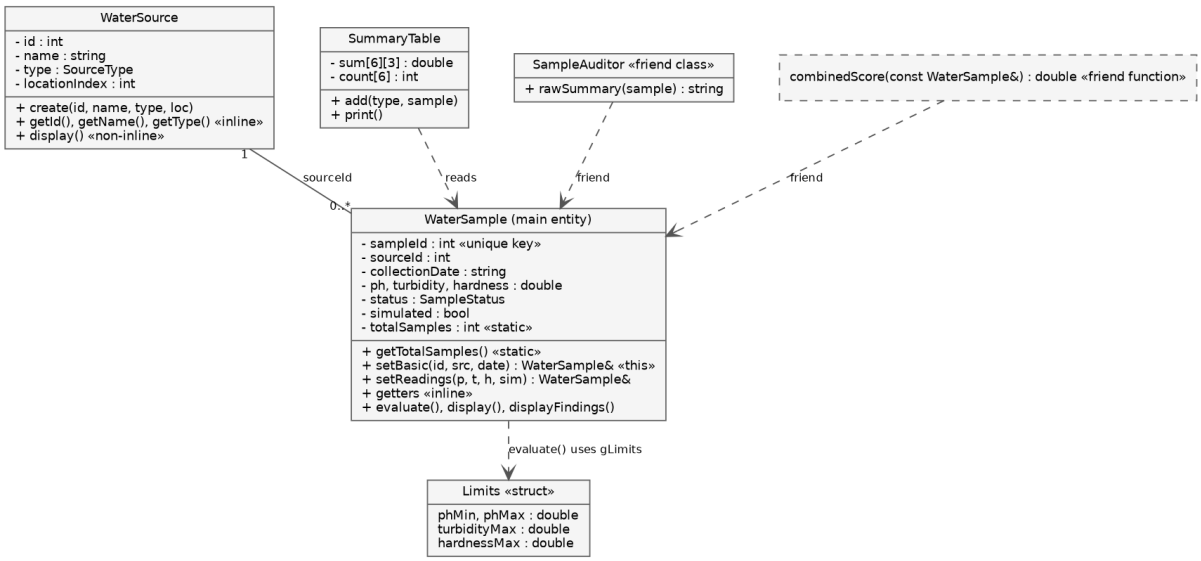


Figure 1: UML diagram of the CA-1 classes

4.5 Program Flowchart

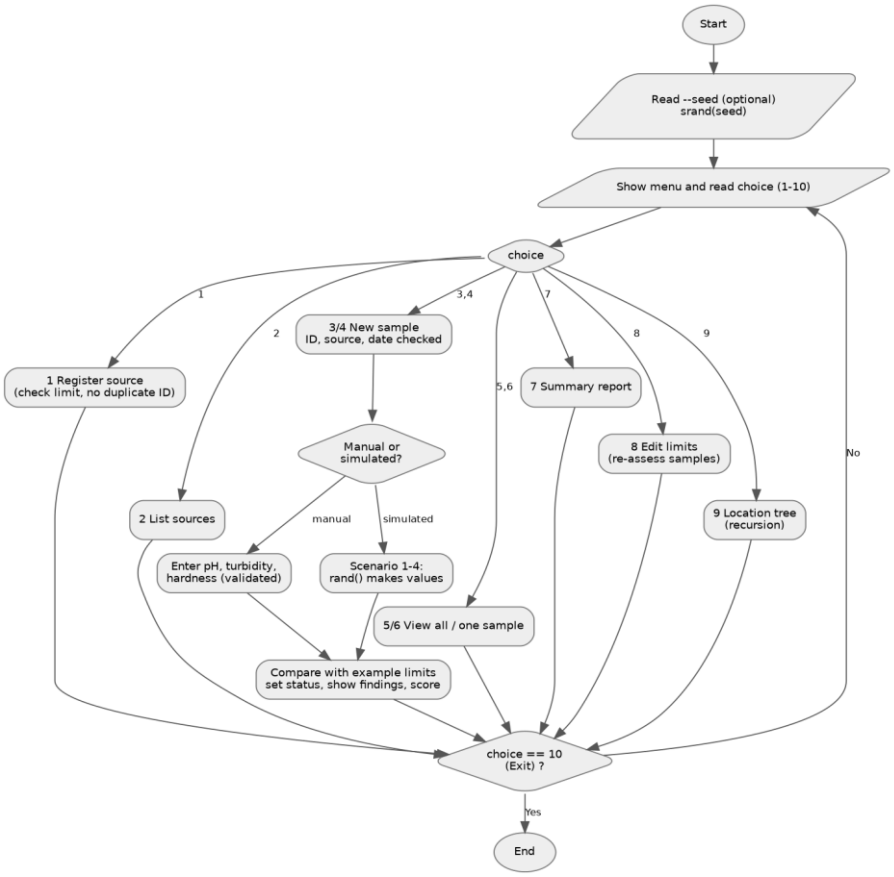


Figure 2: Flow of the menu-driven program

5. Program Working

1. The program starts, reads the optional --seed value for rand(), prints the educational notice and shows the menu.
2. The menu (1 to 10) repeats until the user selects 10: 1 Register source, 2 List sources, 3 New sample (manual), 4 New sample (simulated), 5 View all samples, 6 View one sample, 7 Summary report, 8 Edit example limits, 9 Location tree, 10 Exit.
3. Register source: the program checks the limit of 10 sources, asks for an ID (1 to 999, no duplicates), a name, the type (1 to 5) and a location number from the printed tree.
4. New sample: the program needs at least one source. It asks for a sample ID (no duplicates), a source ID that exists and a valid date.
5. Manual entry asks for pH (0 to 14), turbidity (0 to 1000) and hardness (0 to 2000). Simulated entry asks for a scenario (Typical, After heavy rain, Hard-water area, Fully random) and uses rand(); the sample is labelled SIMULATED.
6. evaluate() compares the readings with the example limits and sets the status; the findings are printed for each parameter together with the combined example score.
7. View all prints one line per sample. View one prints the findings, an audit line and the score. Summary report prints the number of samples created (static counter), flagged readings and averages by source type.

8. Edit limits changes the example limits and re-assesses all stored samples. Location tree prints the location hierarchy using recursion. Exit (10) prints a thank-you message and ends the program.

Sample Run

```
Educational simulation. Example limits only - NOT official safety certification.

========== Community Water Quality Test Manager (CA1) ==========
 1 Register water source    6 View one sample (findings)
 2 List sources             7 Summary report
 3 New sample (manual)      8 Edit example limits
 4 New sample (simulated)   9 Location tree (recursion)
 5 View all samples        10 Exit
Choose: 1

=== Register water source ===
Source ID (1-999): 1
Source name: Village Well
Type: 1 Well, 2 Borewell, 3 Lake, 4 River, 5 Tank
Choose type: 1
Locations:
   - [0] Community
      - [1] North Ward
         - [3] Lake Area
      - [2] South Ward
         - [4] Market Block
         - [5] Old Town
Location number: 3
Source registered.

[main menu shown again]
Choose: 1

=== Register water source ===
Source ID (1-999): 2
Source name: Hand Pump
Type: 1 Well, 2 Borewell, 3 Lake, 4 River, 5 Tank
Choose type: 2
Locations:
   - [0] Community
      - [1] North Ward
         - [3] Lake Area
      - [2] South Ward
         - [4] Market Block
         - [5] Old Town
Location number: 5
Source registered.

[main menu shown again]
Choose: 3

=== New sample (manual readings) ===
Sample ID (1-99999): 1001
Source ID: 1
Collection date (YYYY-MM-DD): 2026-09-15
pH (0-14): 7.2
Turbidity NTU (0-1000): 2.1
Hardness mg/L (0-2000): 180
Sample 1001 collected 2026-09-15  [MANUAL DATA]
  pH        7.20  (example range 6.50-8.50) -> within example limit
  Turbidity 2.10  (example max 5.00 NTU) -> within example limit
  Hardness  180.00  (example max 300.00 mg/L) -> within example limit
  NOTE: these 3 parameters cannot show microbiological safety.
Combined example score: 100.0 / 100

[main menu shown again]
Choose: 3
```

```
=== New sample (manual readings) ===
Sample ID (1-99999): 1001
  Duplicate sample ID. Not saved.

[main menu shown again]
Choose: 3

=== New sample (manual readings) ===
Sample ID (1-99999): 1002
Source ID: 2
Collection date (YYYY-MM-DD): 2026-02-30
  Invalid date. Use YYYY-MM-DD (e.g. 2026-09-15).
Collection date (YYYY-MM-DD): 2026-09-16
pH (0-14): 15
  Invalid. Enter a number from 0.0 to 14.0.
pH (0-14): 5.9
Turbidity NTU (0-1000): 8.0
Hardness mg/L (0-2000): 350
Sample 1002 collected 2026-09-16  [MANUAL DATA]
  pH        5.90  (example range 6.50-8.50) -> REQUIRES ATTENTION
  Turbidity 8.00  (example max 5.00 NTU) -> REQUIRES ATTENTION
  Hardness  350.00  (example max 300.00 mg/L) -> REQUIRES ATTENTION
  NOTE: these 3 parameters cannot show microbiological safety.
Combined example score: 68.0 / 100

[main menu shown again]
Choose: 4

=== New sample (SIMULATED readings) ===
Sample ID (1-99999): 1003
Source ID: 1
Collection date (YYYY-MM-DD): 2026-09-17
Scenario: 1 Typical, 2 After heavy rain, 3 Hard-water area, 4 Fully random
Choose scenario: 2
  (values generated with rand(); they are NOT measurements)
Sample 1003 collected 2026-09-17  [SIMULATED DATA]
  pH        7.10  (example range 6.50-8.50) -> within example limit
  Turbidity 7.50  (example max 5.00 NTU) -> REQUIRES ATTENTION
  Hardness  118.00  (example max 300.00 mg/L) -> within example limit
  NOTE: these 3 parameters cannot show microbiological safety.
Combined example score: 85.0 / 100

[main menu shown again]
Choose: 5

=== All samples ===
  1001 | src   1 | 2026-09-15 | pH  7.2 | Turb   2.1 | Hard    180 | TESTED
  1002 | src   2 | 2026-09-16 | pH  5.9 | Turb   8.0 | Hard    350 | NEEDS ATTENTION
  1003 | src   1 | 2026-09-17 | pH  7.1 | Turb   7.5 | Hard    118 | NEEDS ATTENTION
(simulated)

[main menu shown again]
Choose: 6
Sample ID: 1002
Sample 1002 collected 2026-09-16  [MANUAL DATA]
  pH        5.90  (example range 6.50-8.50) -> REQUIRES ATTENTION
  Turbidity 8.00  (example max 5.00 NTU) -> REQUIRES ATTENTION
  Hardness  350.00  (example max 300.00 mg/L) -> REQUIRES ATTENTION
  NOTE: these 3 parameters cannot show microbiological safety.
audit: id=1002 source=2 status=NEEDS ATTENTION origin=manual
Combined example score: 68.0 / 100

[main menu shown again]
Choose: 7

=== Summary report ===
Samples created so far (static counter): 3
Readings requiring attention: 2 of 3

Averages by source type:
```

```
Type          N      pH      Turb      Hard
Well          2    7.15      4.80    149.00
Borewell      1    5.90      8.00    350.00

Reminder: educational simulation, not a drinking-water certificate.

[main menu shown again]
Choose: 10

Thank you for using the system. Goodbye!
```

6. Explanation of Important Code

6.1 Input validation: readInt() (lines 78 to 87)

```
 78  int readInt(const string& prompt, int lo, int hi) {
 79      int v;
 80      while (true) {
 81          cout << prompt;
 82          if (cin >> v && v >= lo && v <= hi) { cin.ignore(10000, '\n'); return
     v; }
 83          if (cin.eof()) inputEnded();
 84          cin.clear(); cin.ignore(10000, '\n');
 85          cout << "  Invalid. Enter a whole number from " << lo << " to " << hi
     << ".\n";
 86      }
 87  }
```

cin >> v fails when the user types letters, so the if-condition is false and the program continues to the error handling. cin.clear() resets the error state and cin.ignore(10000, '\n') removes the wrong text from the buffer; without both, the loop would repeat forever. Values outside lo to hi are also rejected. If the input ends (Ctrl+D or Ctrl+Z), cin.eof() is true and inputEnded() calls exit(0) instead of looping. readDouble() and readLine() work in the same way, so every input goes through these helper functions.

6.2 Date check: validDate() (lines 109 to 115)

```
109  bool validDate(const string& d) {                       // YYYY-MM-DD
110      if (d.size() != 10 || d[4] != '-' || d[7] != '-') return false;
111      for (int i = 0; i < 10; i++) if (i != 4 && i != 7 && !isdigit((unsigned
     char)d[i])) return false;
112      int y = stoi(d.substr(0, 4)), m = stoi(d.substr(5, 2)), day =
     stoi(d.substr(8, 2));
113      int dim[] = {31, isLeap(y) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
     31};
114      return y >= 2000 && y <= 2100 && m >= 1 && m <= 12 && day >= 1 && day <=
     dim[m - 1];
115  }
```

The function first checks the length and the two dashes, then that all other characters are digits. stoi() converts year, month and day. The array dim[] holds the days of each month, and isLeap(y) puts 29 or 28 for February, so 2028-02-29 is accepted but 2027-02-29 and 2026-02-30 are rejected.

6.3 Entity class: static member and this pointer (lines 152 to 168)

```
152  class WaterSample {
153      int sampleId = 0;
154      int sourceId = 0;
155      string collectionDate;
156      double ph = 7.0, turbidity = 0.0, hardness = 0.0;
157      SampleStatus status = PENDING;
158      bool simulated = false;           // true when values came from rand()
```

```
159      static int totalSamples;          // Unit I: static data member
160  public:
161      static int getTotalSamples() { return totalSamples; }   // static member
     function
162
163      // 'this' pointer: each setter returns *this so calls can be chained
164      WaterSample& setBasic(int id, int srcId, const string& date) {
165          this->sampleId = id; this->sourceId = srcId; this->collectionDate =
     date;
166          totalSamples++;               // one more sample created
167          return *this;
168      }
```

The data members are private. totalSamples is static, so one copy is shared by all objects; it is defined once outside the class (line 200) and increased in setBasic() each time a new sample is created. setBasic() and setReadings() use the this pointer and return *this, so line 331 can chain them: samples[sampleCount].setBasic(id, src, date).setReadings(ph, turb, hard, simulate). Getters defined inside the class are inline functions; display() and evaluate() are defined outside the class (non-inline).

6.4 Evaluation and friend function (lines 202 to 206 and 226 to 233)

```
202  void WaterSample::evaluate() {
203      bool bad = ph < gLimits.phMin || ph > gLimits.phMax ||
204                 turbidity > gLimits.turbidityMax || hardness >
     gLimits.hardnessMax;
205      status = bad ? ATTENTION : TESTED;
206  }


226  double combinedScore(const WaterSample& s) {
227      double score = 100.0;
228      if (s.ph < gLimits.phMin) score -= (gLimits.phMin - s.ph) * 15;
229      if (s.ph > gLimits.phMax) score -= (s.ph - gLimits.phMax) * 15;
230      if (s.turbidity > gLimits.turbidityMax) score -= (s.turbidity -
     gLimits.turbidityMax) * 6;
231      if (s.hardness > gLimits.hardnessMax) score -= (s.hardness -
     gLimits.hardnessMax) / 10.0;
232      return score < 0 ? 0 : score;
233  }
```

evaluate() sets the status to ATTENTION if any reading is outside the example limits stored in gLimits, so changing the limits changes the result. combinedScore() is a friend function: it is not a member, but the class allows it to read the private readings. It starts at 100 and subtracts a penalty for every reading outside its limit (the weights 15, 6 and 1/10 are only example numbers for learning). The score is shown next to the findings and is not a certificate.

6.5 Call by reference and address: simulateReadings() (lines 297 to 309)

```
297  void clampToRange(double* value, double lo, double hi) {      // call by
     ADDRESS
298      if (*value < lo) *value = lo;
299      if (*value > hi) *value = hi;
300  }
301  void simulateReadings(int scenario, double& ph, double& turb, double& hard) {
     // call by REFERENCE
302      switch (scenario) {
303          case 1: ph = 6.8 + (rand() % 15) / 10.0; turb = (rand() % 40) / 10.0;
     hard = 80 + rand() % 200; break;   // typical
304          case 2: ph = 6.5 + (rand() % 20) / 10.0; turb = 5 + (rand() % 60) /
     10.0; hard = 100 + rand() % 150; break; // after rain
305          case 3: ph = 7.0 + (rand() % 20) / 10.0; turb = (rand() % 30) / 10.0;
     hard = 300 + rand() % 200; break;  // hard water
306          default: ph = 4.0 + (rand() % 90) / 10.0; turb = (rand() % 100) /
     10.0; hard = 20 + rand() % 480; break; // fully random
```

```
307      }
308      clampToRange(&ph, 0, 14);
309  }
```

simulateReadings() receives ph, turb and hard by reference, so it can give three values back to addSample() without return. A switch on the scenario number chooses the value ranges, and rand() % n gives the random part. clampToRange() receives an address (double*) and changes the caller's variable through *value, so a pH can never go outside 0 to 14.

6.6 Recursion: printLocationTree() (lines 59 to 67)

```
59  void printLocationTree(int node, int depth) {          // recursive function
60      cout << string(depth * 3, ' ') << "- [" << node << "] " <<
    locations[node].name << "\n";
61      for (int i = 0; i < LOCATION_COUNT; i++)
62          if (locations[i].parent == node) printLocationTree(i, depth + 1);
63  }
64  string locationPath(int node) {                         // recursion that
    returns a value
65      if (locations[node].parent == -1) return locations[node].name;
66      return locationPath(locations[node].parent) + " > " +
    locations[node].name;
67  }
```

Locations form a tree: Community has North Ward and South Ward, which have areas. printLocationTree() prints one node, indented by depth, and calls itself for every child. locationPath() calls itself for the parent and builds a path like Community > South Ward > Market Block; its stopping condition (base case) is the root with parent -1.

6.7 Pointers and arrays of objects (lines 265 to 269 and 331 to 335)

```
265  WaterSample* findSample(int id) {              // returns pointer or nullptr
266      for (int i = 0; i < sampleCount; i++)
267          if (samples[i].getId() == id) return &samples[i];
268      return nullptr;
269  }
```

```
331      samples[sampleCount].setBasic(id, src, date).setReadings(ph, turb, hard,
    simulate);
332      samples[sampleCount].displayFindings();
333      cout << "Combined example score: " << fixed << setprecision(1)
334           << combinedScore(samples[sampleCount]) << " / 100\n";
335      sampleCount++;
```

findSample() walks the array of objects and returns the address of the matching sample, or nullptr when there is none, so callers such as viewOne() and addSample() test the pointer before using it. New samples are stored in samples[sampleCount]; the check sampleCount >= MAX_SAMPLES at the start of addSample() prevents writing outside the array.

7. Testing and Expected Results

The program was compiled with g++ in C++11, C++14, C++17 and C++20 modes using -Wall -Wextra -pedantic with no warnings, and run with AddressSanitizer and UndefinedBehaviorSanitizer with no memory errors. The following cases were tested (inputs are listed in the project folder tests/ and can be repeated with bash tests/run_tests.sh):

#	Test case / Input	Expected result	Actual	Status
1	Register source ID 1, then register ID 1 again	Second one rejected: Duplicate source ID. Not saved.	Same	Pass
2	Sample 1001, pH 7.2, turbidity 2.1, hardness 180	All three within example limit; score 100.0 / 100	Same	Pass
3	Sample ID 1001 entered again	Duplicate sample ID. Not saved.	Same	Pass
4	Date 2026-02-30	Invalid date message, asks again	Same	Pass
5	pH 15, then letters and -1 as pH	Asks again: Enter a number from 0.0 to 14.0.	Same	Pass
6	Menu input abc, 99 and -1	Asks again: Enter a whole number from 1 to 10.	Same	Pass
7	Sample with source ID 99 (not registered)	Source not found.	Same	Pass
8	New sample before any source is registered	Register a source first.	Same	Pass
9	Sample 1002: pH 5.9, turbidity 8.0, hardness 350	Three readings REQUIRES ATTENTION; status NEEDS ATTENTION; score 68.0	Same	Pass
10	Simulated sample, scenario 2, --seed 3	Labelled SIMULATED DATA; same values on every run with the same seed	Same	Pass
11	Register an 11th source	Source limit reached.	Same	Pass
12	Add a 51st sample	Sample limit reached. (50 stored)	Same	Pass
13	Dates 2028-02-29 and 2027-02-29	First accepted (leap year), second rejected	Same	Pass
14	Edit limits: pH minimum 7.0, maximum 6.0	Maximum below minimum is rejected, asks again	Same	Pass
15	Summary after 3 samples (2 flagged)	Samples created 3; Readings requiring attention: 2 of 3; averages by type	Same	Pass
16	List sources after registering two sources	Locations shown as Community > North Ward > Lake Area (recursive path)	Same	Pass
17	Input stream closed (Ctrl+D / Ctrl+Z) during input	Program exits with Input ended. Exiting. instead of looping	Same	Pass
18	Menu option 10 (Exit)	Thank you for using the system. Goodbye! and the program ends	Same	Pass

Invalid input test (cases 3, 4 and 5)

```
Sample ID (1-99999): 1001
  Duplicate sample ID. Not saved.
...
Collection date (YYYY-MM-DD): 2026-02-30
  Invalid date. Use YYYY-MM-DD (e.g. 2026-09-15).
Collection date (YYYY-MM-DD): 2026-09-16
pH (0-14): 15
  Invalid. Enter a number from 0.0 to 14.0.
pH (0-14): 5.9
```

8. Current Scope and Limitations

Current scope

- Registering up to 10 water sources and 50 samples in memory, with unique IDs and valid dates.
- Manual or rand() simulated readings of pH, turbidity and hardness, compared with editable example limits.
- Parameter-wise findings, a combined example score, a list of samples and a summary report.
- Full input validation for every entry.

Limitations

- Data is kept only in memory and is lost when the program closes (file saving is planned for CA-2).
- Edited limits are also not saved; there is no update or delete of a sample yet.
- Objects are filled with setBasic() and setReadings(); constructors and destructors come in CA-2.
- Only three parameters. They cannot show microbiological or chemical safety, and the example limits and the score weights are not official.
- Capacity is fixed by the arrays (10 sources, 50 samples); no search by date or status yet.
- The pending status is only the starting value; samples are tested as soon as they are entered.
- rand() values are simulated data, not measurements.

9. Future Enhancements

The project is planned to grow in the next three CAs using the same logic:

CA	Planned enhancement
CA-2	Constructors (default, parameterized with default arguments, copy, initializer list) and a destructor. Save samples and sources in binary files with fixed-size records so data survives a restart; search with seekg and update with seekp.
CA-3	An abstract WaterTest base class with PHTest, TurbidityTest and HardnessTest using a virtual evaluate(); operator overloading (<<, >>, ==, <) and type conversion; new/delete and a linked pending queue; safe update and delete with confirmation.
CA-4	Custom exception classes with try, catch and re-throw; function and class templates; STL vector and list with sort, find_if and count_if; search and filter; backup, CSV export and undo; final report.

10. Appendix A: Source Code

File: main.cpp (line numbers added for reference)

```
 1  // ========================================================================
 2  // CAP455 Minor Project - CA1 (Design & Create)       Units I - II
 3  // Community Water Quality Test Manager (educational simulation)
 4  //
 5  // DISCLAIMER: Limits used here are CONFIGURABLE EXAMPLE values only.
 6  // This program is NOT an official water-quality certificate. Real
 7  // drinking-water safety needs approved standards, validated methods
 8  // and a qualified laboratory (including microbiological tests).
 9  //
10  // Build:  g++ -std=c++17 -Wall -Wextra -o ca1 main.cpp
11  // Run  :  ./ca1            (optional: ./ca1 --seed 7  for repeatable rand())
12  // ========================================================================
13  #include <iostream>
```

```
14  #include <iomanip>
15  #include <string>
16  #include <cstdlib>   // rand, srand, exit
17  #include <cctype>
18  #include <ctime>
19  using namespace std;
20
21  // ---------- Unit I : enumerations ----------
22  enum SourceType { WELL = 1, BOREWELL, LAKE, RIVER, TANK };
23  enum SampleStatus { PENDING, TESTED, ATTENTION };
24
25  const int MAX_SOURCES = 10;
26  const int MAX_SAMPLES = 50;
27
28  const char* sourceTypeName(SourceType t) {
29      static const char* names[] = {"?", "Well", "Borewell", "Lake", "River",
    "Tank"};
30      return names[t];
31  }
32  const char* statusName(SampleStatus s) {
33      static const char* names[] = {"PENDING", "TESTED", "NEEDS ATTENTION"};
34      return names[s];
35  }
36
37  // ---------- Unit I : structure (all members live together) ----------
38  // Configurable EXAMPLE reference limits (not official thresholds).
39  struct Limits {
40      double phMin = 6.5;
41      double phMax = 8.5;
42      double turbidityMax = 5.0;   // NTU (example)
43      double hardnessMax = 300.0;  // mg/L as CaCO3 (example)
44  };
45  Limits gLimits;   // one shared set of limits for the program
46
47  // ---------- Unit I : recursion - hierarchical list of locations ----------
48  struct Location { const char* name; int parent; };
49  Location locations[] = {
50      {"Community", -1},   // 0
51      {"North Ward", 0},   // 1
52      {"South Ward", 0},   // 2
53      {"Lake Area", 1},    // 3
54      {"Market Block", 2}, // 4
55      {"Old Town", 2}      // 5
56  };
57  const int LOCATION_COUNT = sizeof(locations) / sizeof(locations[0]);
58
59  void printLocationTree(int node, int depth) {          // recursive function
60      cout << string(depth * 3, ' ') << "- [" << node << "] " <<
    locations[node].name << "\n";
61      for (int i = 0; i < LOCATION_COUNT; i++)
62          if (locations[i].parent == node) printLocationTree(i, depth + 1);
63  }
64  string locationPath(int node) {                          // recursion that
    returns a value
65      if (locations[node].parent == -1) return locations[node].name;
66      return locationPath(locations[node].parent) + " > " +
    locations[node].name;
67  }
68
69  // ---------- Unit I : function overloading ----------
70  void banner(const string& title) { cout << "\n=== " << title << " ===\n"; }
71  void banner(const string& title, int width) {
72      cout << "\n" << string(width, '=') << "\n " << title << "\n" <<
    string(width, '=') << "\n";
73  }
74
75  // ---------- Safe input helpers (no infinite loops: stop on end-of-input) ---
    -------
76  void inputEnded() { cout << "\nInput ended. Exiting.\n"; exit(0); }
```

```
77
78  int readInt(const string& prompt, int lo, int hi) {
79      int v;
80      while (true) {
81          cout << prompt;
82          if (cin >> v && v >= lo && v <= hi) { cin.ignore(10000, '\n'); return
     v; }
83          if (cin.eof()) inputEnded();
84          cin.clear(); cin.ignore(10000, '\n');
85          cout << "  Invalid. Enter a whole number from " << lo << " to " << hi
     << ".\n";
86      }
87  }
88  double readDouble(const string& prompt, double lo, double hi) {
89      double v;
90      while (true) {
91          cout << prompt;
92          if (cin >> v && v >= lo && v <= hi) { cin.ignore(10000, '\n'); return
     v; }
93          if (cin.eof()) inputEnded();
94          cin.clear(); cin.ignore(10000, '\n');
95          cout << "  Invalid. Enter a number from " << lo << " to " << hi <<
     ".\n";
96      }
97  }
98  string readLine(const string& prompt) {
99      string s;
100     while (true) {
101         cout << prompt;
102         getline(cin, s);
103         if (cin.eof() && s.empty()) inputEnded();
104         if (!s.empty()) return s;
105         cout << "  Text cannot be empty.\n";
106     }
107 }
108 bool isLeap(int y) { return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; }
109 bool validDate(const string& d) {                       // YYYY-MM-DD
110     if (d.size() != 10 || d[4] != '-' || d[7] != '-') return false;
111     for (int i = 0; i < 10; i++) if (i != 4 && i != 7 && !isdigit((unsigned
     char)d[i])) return false;
112     int y = stoi(d.substr(0, 4)), m = stoi(d.substr(5, 2)), day =
     stoi(d.substr(8, 2));
113     int dim[] = {31, isLeap(y) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
     31};
114     return y >= 2000 && y <= 2100 && m >= 1 && m <= 12 && day >= 1 && day <=
     dim[m - 1];
115 }
116 string readDate(const string& prompt) {
117     while (true) {
118         string d = readLine(prompt);
119         if (validDate(d)) return d;
120         cout << "  Invalid date. Use YYYY-MM-DD (e.g. 2026-09-15).\n";
121     }
122 }
123
124 // ============================================================================
125 // Class WaterSource  (Unit I: classes & objects, inline / non-inline)
126 // ============================================================================
127 class WaterSource {
128     int id = 0;
129     string name;
130     SourceType type = WELL;
131     int locationIndex = 0;
132 public:
133     void create(int i, const string& n, SourceType t, int loc) {
134         id = i; name = n; type = t; locationIndex = loc;
135     }
136     // inline member functions (defined inside the class)
137     int getId() const { return id; }
```

```
138      const string& getName() const { return name; }
139      SourceType getType() const { return type; }
140      // non-inline member function (declared here, defined outside)
141      void display() const;
142  };
143  void WaterSource::display() const {
144      cout << setw(4) << id << " | " << left << setw(14) << name << right
145           << " | " << setw(8) << sourceTypeName(type) << " | " <<
     locationPath(locationIndex) << "\n";
146  }
147
148  // ============================================================================
149  // Class WaterSample  (entity class: static, inline, friend, this, string)
150  // Key = sampleId (must be unique)
151  // ============================================================================
152  class WaterSample {
153      int sampleId = 0;
154      int sourceId = 0;
155      string collectionDate;
156      double ph = 7.0, turbidity = 0.0, hardness = 0.0;
157      SampleStatus status = PENDING;
158      bool simulated = false;           // true when values came from rand()
159      static int totalSamples;          // Unit I: static data member
160  public:
161      static int getTotalSamples() { return totalSamples; }   // static member
     function
162
163      // 'this' pointer: each setter returns *this so calls can be chained
164      WaterSample& setBasic(int id, int srcId, const string& date) {
165          this->sampleId = id; this->sourceId = srcId; this->collectionDate =
     date;
166          totalSamples++;               // one more sample created
167          return *this;
168      }
169      WaterSample& setReadings(double p, double t, double h, bool isSim) {
170          this->ph = p; this->turbidity = t; this->hardness = h; this->simulated
     = isSim;
171          status = TESTED;
172          evaluate();
173          return *this;
174      }
175      // inline getters
176      int getId() const { return sampleId; }
177      int getSourceId() const { return sourceId; }
178      double getPh() const { return ph; }
179      double getTurbidity() const { return turbidity; }
180      double getHardness() const { return hardness; }
181      SampleStatus getStatus() const { return status; }
182      bool isSimulated() const { return simulated; }
183
184      // non-inline members (defined below)
185      void evaluate();
186      void display() const;
187      void displayFindings() const;
188
189      friend class SampleAuditor;                          // friend class
190      friend double combinedScore(const WaterSample& s);   // Unit I: friend
     function
191  };
192  // Unit I: FRIEND CLASS - every member function of SampleAuditor may read
     WaterSample's private data
193  class SampleAuditor {
194  public:
195      static string rawSummary(const WaterSample& s) {
196          return "audit: id=" + to_string(s.sampleId) + " source=" +
     to_string(s.sourceId) +
197                 " status=" + statusName(s.status) + (s.simulated ? "
     origin=simulated" : " origin=manual");
198      }
```

```
199 };
200 int WaterSample::totalSamples = 0;   // static member must be defined once
    outside
201
202 void WaterSample::evaluate() {
203     bool bad = ph < gLimits.phMin || ph > gLimits.phMax ||
204                turbidity > gLimits.turbidityMax || hardness >
    gLimits.hardnessMax;
205     status = bad ? ATTENTION : TESTED;
206 }
207 void WaterSample::display() const {
208     cout << setw(6) << sampleId << " | src " << setw(3) << sourceId << " | "
    << collectionDate
209          << " | pH " << fixed << setprecision(1) << setw(4) << ph
210          << " | Turb " << setw(5) << turbidity << " | Hard " << setw(6) <<
    setprecision(0) << hardness
211          << " | " << statusName(status) << (simulated ? " (simulated)" : "")
    << "\n";
212 }
213 void WaterSample::displayFindings() const {
214     cout << "Sample " << sampleId << " collected " << collectionDate
215          << (simulated ? "  [SIMULATED DATA]" : "  [MANUAL DATA]") << "\n" <<
    fixed << setprecision(2);
216     cout << "  pH        " << ph << "  (example range " << gLimits.phMin << "-
    " << gLimits.phMax << ") -> "
217          << ((ph < gLimits.phMin || ph > gLimits.phMax) ? "REQUIRES ATTENTION"
    : "within example limit") << "\n";
218     cout << "  Turbidity " << turbidity << "  (example max " <<
    gLimits.turbidityMax << " NTU) -> "
219          << (turbidity > gLimits.turbidityMax ? "REQUIRES ATTENTION" : "within
    example limit") << "\n";
220     cout << "  Hardness  " << hardness << "  (example max " <<
    gLimits.hardnessMax << " mg/L) -> "
221          << (hardness > gLimits.hardnessMax ? "REQUIRES ATTENTION" : "within
    example limit") << "\n";
222     cout << "  NOTE: these 3 parameters cannot show microbiological
    safety.\n";
223 }
224
225 // friend function: reads private data of WaterSample to compute a 0-100 score
226 double combinedScore(const WaterSample& s) {
227     double score = 100.0;
228     if (s.ph < gLimits.phMin) score -= (gLimits.phMin - s.ph) * 15;
229     if (s.ph > gLimits.phMax) score -= (s.ph - gLimits.phMax) * 15;
230     if (s.turbidity > gLimits.turbidityMax) score -= (s.turbidity -
    gLimits.turbidityMax) * 6;
231     if (s.hardness > gLimits.hardnessMax) score -= (s.hardness -
    gLimits.hardnessMax) / 10.0;
232     return score < 0 ? 0 : score;
233 }
234
235 // ========================================================================
236 // Class SummaryTable : multidimensional array INSIDE a class (Unit II)
237 // ========================================================================
238 class SummaryTable {
239     double sum[6][3] = {};   // [source type][pH, turbidity, hardness]
240     int count[6] = {};
241 public:
242     void add(SourceType t, const WaterSample& s) {
243         sum[t][0] += s.getPh(); sum[t][1] += s.getTurbidity(); sum[t][2] +=
    s.getHardness();
244         count[t]++;
245     }
246     void print() const {
247         cout << left << setw(10) << "Type" << right << setw(5) << "N" <<
    setw(8) << "pH"
248              << setw(10) << "Turb" << setw(10) << "Hard" << "\n" << fixed <<
    setprecision(2);
249         for (int t = WELL; t <= TANK; t++) {
```

```
250             if (count[t] == 0) continue;
251             cout << left << setw(10) << sourceTypeName((SourceType)t) << right
    << setw(5) << count[t];
252             for (int p = 0; p < 3; p++) cout << setw(p == 0 ? 8 : 10) <<
    sum[t][p] / count[t];
253             cout << "\n";
254         }
255     }
256 };
257
258 // ---------- Program data (Unit II: arrays of objects) ----------
259 WaterSource sources[MAX_SOURCES];
260 int sourceCount = 0;
261 WaterSample samples[MAX_SAMPLES];
262 int sampleCount = 0;
263
264 // ---------- Unit II: pointers used in the real application ----------
265 WaterSample* findSample(int id) {               // returns pointer or nullptr
266     for (int i = 0; i < sampleCount; i++)
267         if (samples[i].getId() == id) return &samples[i];
268     return nullptr;
269 }
270 WaterSource* findSource(int id) {
271     for (int i = 0; i < sourceCount; i++)
272         if (sources[i].getId() == id) return &sources[i];
273     return nullptr;
274 }
275
276 // ---------- Registering ----------
277 void registerSource() {
278     banner("Register water source");
279     if (sourceCount >= MAX_SOURCES) { cout << "Source limit reached.\n";
    return; }
280     int id = readInt("Source ID (1-999): ", 1, 999);
281     if (findSource(id)) { cout << "  Duplicate source ID. Not saved.\n";
    return; }
282     string name = readLine("Source name: ");
283     cout << "Type: 1 Well, 2 Borewell, 3 Lake, 4 River, 5 Tank\n";
284     int t = readInt("Choose type: ", 1, 5);
285     cout << "Locations:\n"; printLocationTree(0, 1);
286     int loc = readInt("Location number: ", 0, LOCATION_COUNT - 1);
287     sources[sourceCount++].create(id, name, (SourceType)t, loc);
288     cout << "Source registered.\n";
289 }
290 void listSources() {
291     banner("Registered sources");
292     if (sourceCount == 0) { cout << "None yet.\n"; return; }
293     for (int i = 0; i < sourceCount; i++) sources[i].display();
294 }
295
296 // Unit I: call by value / address / reference used for real work
297 void clampToRange(double* value, double lo, double hi) {      // call by
    ADDRESS
298     if (*value < lo) *value = lo;
299     if (*value > hi) *value = hi;
300 }
301 void simulateReadings(int scenario, double& ph, double& turb, double& hard) {
    // call by REFERENCE
302     switch (scenario) {
303         case 1: ph = 6.8 + (rand() % 15) / 10.0; turb = (rand() % 40) / 10.0;
    hard = 80 + rand() % 200; break;   // typical
304         case 2: ph = 6.5 + (rand() % 20) / 10.0; turb = 5 + (rand() % 60) /
    10.0; hard = 100 + rand() % 150; break; // after rain
305         case 3: ph = 7.0 + (rand() % 20) / 10.0; turb = (rand() % 30) / 10.0;
    hard = 300 + rand() % 200; break;  // hard water
306         default: ph = 4.0 + (rand() % 90) / 10.0; turb = (rand() % 100) /
    10.0; hard = 20 + rand() % 480; break; // fully random
307     }
308     clampToRange(&ph, 0, 14);
```

```
}

void addSample(bool simulate) {
    banner(simulate ? "New sample (SIMULATED readings)" : "New sample (manual readings)");
    if (sourceCount == 0) { cout << "Register a source first.\n"; return; }
    if (sampleCount >= MAX_SAMPLES) { cout << "Sample limit reached.\n"; return; }
    int id = readInt("Sample ID (1-99999): ", 1, 99999);
    if (findSample(id)) { cout << "  Duplicate sample ID. Not saved.\n"; return; }
    int src = readInt("Source ID: ", 1, 999);
    if (!findSource(src)) { cout << "  Source not found.\n"; return; }
    string date = readDate("Collection date (YYYY-MM-DD): ");
    double ph, turb, hard;
    if (simulate) {
        cout << "Scenario: 1 Typical, 2 After heavy rain, 3 Hard-water area, 4 Fully random\n";
        int sc = readInt("Choose scenario: ", 1, 4);
        simulateReadings(sc, ph, turb, hard);
        cout << "  (values generated with rand(); they are NOT measurements)\n";
    } else {
        ph = readDouble("pH (0-14): ", 0, 14);
        turb = readDouble("Turbidity NTU (0-1000): ", 0, 1000);
        hard = readDouble("Hardness mg/L (0-2000): ", 0, 2000);
    }
    samples[sampleCount].setBasic(id, src, date).setReadings(ph, turb, hard, simulate);
    samples[sampleCount].displayFindings();
    cout << "Combined example score: " << fixed << setprecision(1)
         << combinedScore(samples[sampleCount]) << " / 100\n";
    sampleCount++;
}

void viewAll() {
    banner("All samples");
    if (sampleCount == 0) { cout << "None yet.\n"; return; }
    for (int i = 0; i < sampleCount; i++) samples[i].display();
}
void viewOne() {
    int id = readInt("Sample ID: ", 1, 99999);
    WaterSample* p = findSample(id);
    if (!p) { cout << "  Sample not found.\n"; return; }
    p->displayFindings();
    cout << SampleAuditor::rawSummary(*p) << "\n";
    cout << "Combined example score: " << fixed << setprecision(1) << combinedScore(*p) << " / 100\n";
}
void summaryReport() {
    banner("Summary report");
    int stat[3][6] = {};   // multidimensional array in main-level code: [status][source type]
    SummaryTable table;
    for (int i = 0; i < sampleCount; i++) {
        WaterSource* src = findSource(samples[i].getSourceId());
        SourceType t = src ? src->getType() : WELL;
        stat[samples[i].getStatus()][t]++;
        table.add(t, samples[i]);
    }
    cout << "Samples created so far (static counter): " << WaterSample::getTotalSamples() << "\n";
    int needs = 0;
    for (int t = WELL; t <= TANK; t++) needs += stat[ATTENTION][t];
    cout << "Readings requiring attention: " << needs << " of " << sampleCount << "\n";
    if (sampleCount) { cout << "\nAverages by source type:\n"; table.print(); }
}
```

```
366      cout << "\nReminder: educational simulation, not a drinking-water
     certificate.\n";
367  }
368  void editLimits() {
369      banner("Edit example reference limits");
370      gLimits.phMin = readDouble("pH minimum (0-14): ", 0, 14);
371      gLimits.phMax = readDouble("pH maximum (0-14): ", gLimits.phMin, 14);
372      gLimits.turbidityMax = readDouble("Turbidity max NTU (0-1000): ", 0,
     1000);
373      gLimits.hardnessMax = readDouble("Hardness max mg/L (0-2000): ", 0, 2000);
374      for (int i = 0; i < sampleCount; i++) samples[i].evaluate();   // re-
     assess old samples
375      cout << "Limits updated; existing samples re-assessed.\n";
376  }
377
378  void menu() {
379      cout << "\n========== Community Water Quality Test Manager (CA1)
     ==========\n"
380           << " 1 Register water source    6 View one sample (findings)\n"
381           << " 2 List sources             7 Summary report\n"
382           << " 3 New sample (manual)      8 Edit example limits\n"
383           << " 4 New sample (simulated)   9 Location tree (recursion)\n"
384           << " 5 View all samples        10 Exit\n";
385
386  }
387
388  int main(int argc, char* argv[]) {
389      unsigned seed = (unsigned)time(nullptr);
390      if (argc == 3 && string(argv[1]) == "--seed") seed =
     (unsigned)atoi(argv[2]);
391      srand(seed);
392      cout << "Educational simulation. Example limits only - NOT official safety
     certification.\n";
393      int choice;
394      do {
395          menu();
396          choice = readInt("Choose: ", 1, 10);
397          switch (choice) {
398              case 1: registerSource(); break;
399              case 2: listSources(); break;
400              case 3: addSample(false); break;
401              case 4: addSample(true); break;
402              case 5: viewAll(); break;
403              case 6: viewOne(); break;
404              case 7: summaryReport(); break;
405              case 8: editLimits(); break;
406              case 9: banner("Location hierarchy", 40); printLocationTree(0, 0);
     break;
407              case 10: cout << "\nThank you for using the system. Goodbye!\n";
     break;
408          }
409      } while (choice != 10);
410      return 0;
411  }
```

11. Conclusion

The CA-1 increment of the Community Water Quality Test Manager was completed as a working, menu-driven C++ console program. It registers water sources and samples with unique keys and valid dates, accepts manual or simulated readings, compares them with configurable example limits and shows parameter-wise findings, a combined example score and a summary report.

All requirements of CA-1 were met with Unit I and Unit II features: an entity class with static, inline and friend members, enumerations, structures, references and pointers, arrays and arrays of objects,

strings and recursion. The program compiles without warnings, validates every input and passed all test cases. It remains an educational simulation and not a drinking-water certificate.

Since the data is already grouped in classes and the logic is divided into small functions, the program is ready to get constructors and file handling in CA-2, inheritance and polymorphism in CA-3, and exceptions, templates and the STL in CA-4.